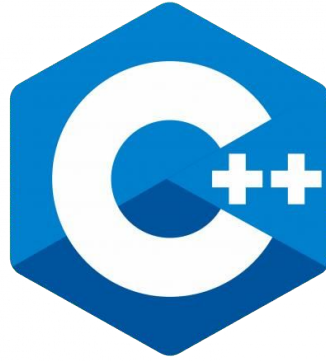




C++ Programming

Trainer : Priyanka R. Rangole

Email: priyanka.rangole@sunbeaminfo.com



Inheritance

- If "is-a" relationship exist between two types then we should use inheritance.
- Inheritance is also called as " journey from Generalization to Specialization".
- Example: Book is-a product
- During inheritance, members of base class inherit into derived class.
- If we create object of derived class then non static data members declared in base class get space inside it.
- Size of object = sum of size of non static data members declared in base class and derived class.
- If we use private/protected/public keyword to control visibility of members of class by using access Specifier.
- If we use private/protected/public keyword to extend the class then it is called mode of inheritance.
- Default mode of inheritance is private.
 - Example: class Employee : person //is treated as class Employee : private Person
- Example: class Employee:public Person
- In all types of mode, private members inherit into derived class but we can not access it inside member function of derived class.



Syntax of inheritance in C++

<pre>class Person //Parent class { }; class Employee : public Person // Child class { };</pre>	In C++ Parent class is called as Base class and child class is called as derived class. To create derived class we should use colon(:) operator. As shown in this code, public is mode of inheritance.
<pre>class Person //Parent class { char name[30]; int age; };</pre> <pre>class Employee : public Person //Child class { int empid; float salary; };</pre> <pre>int main(void) { Person p; cout<<sizeof(p)<<endl; Employee emp; cout<<sizeof(emp)<<endl; return 0; }</pre>	If we create object of derived class, then all the non- static data member declared in base class & derived class get space inside it i.e. non-static. static data members of base class inherit into the derived class.



Except following functions, including nested class, all the members of base class, inherit into the derived class

- Constructor
- Destructor
- Copy constructor
- Assignment operator
- Friend function.



Protected Data member

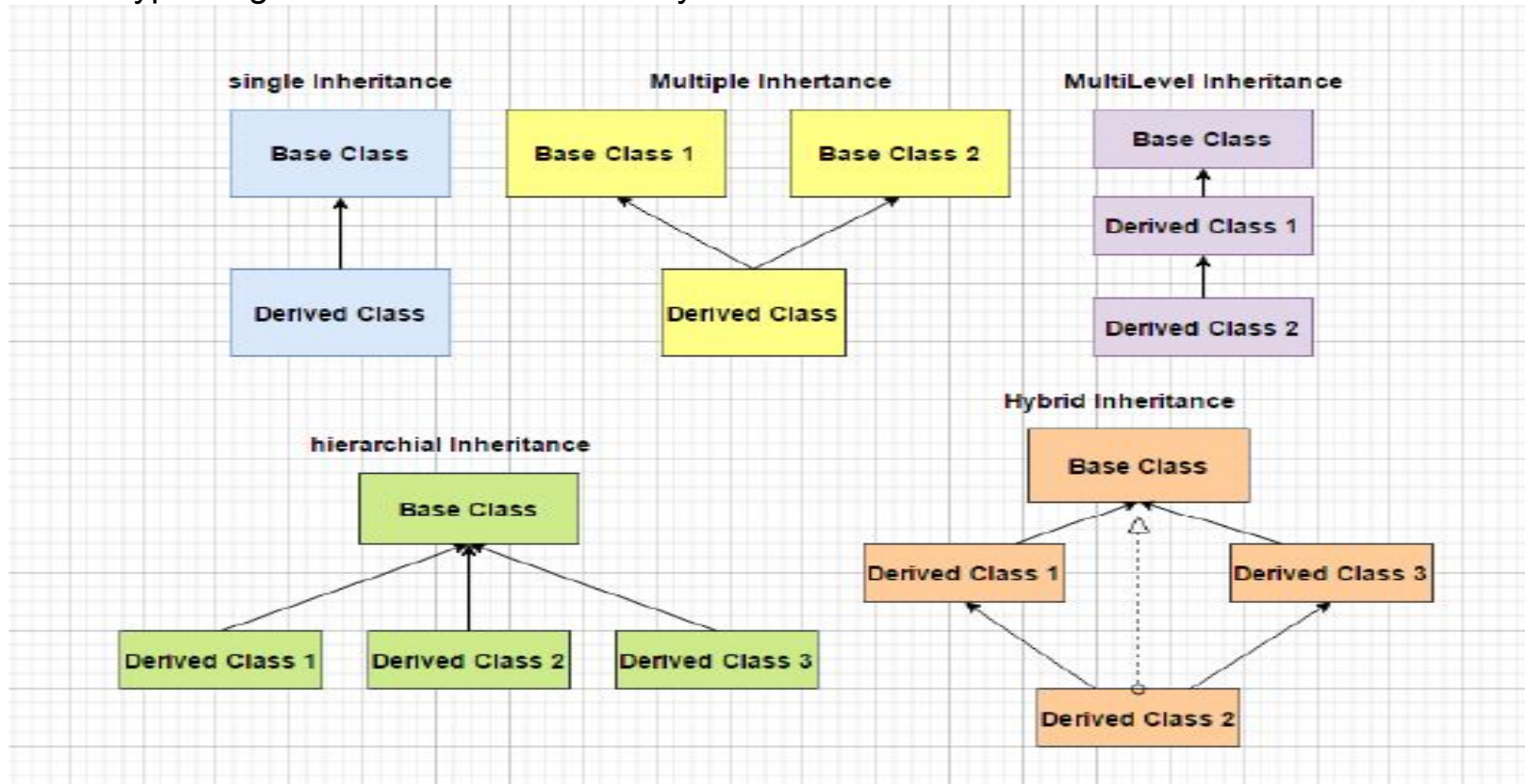
- The protected access specifier allows the base class members to access onto derived class.
- However, protected members are not accessible from outside the class and global functions like main().
- Protected members in a class are similar to private members as they cannot be accessed from outside the class.
- But they can be accessed by derived classes or child classes while private members cannot.



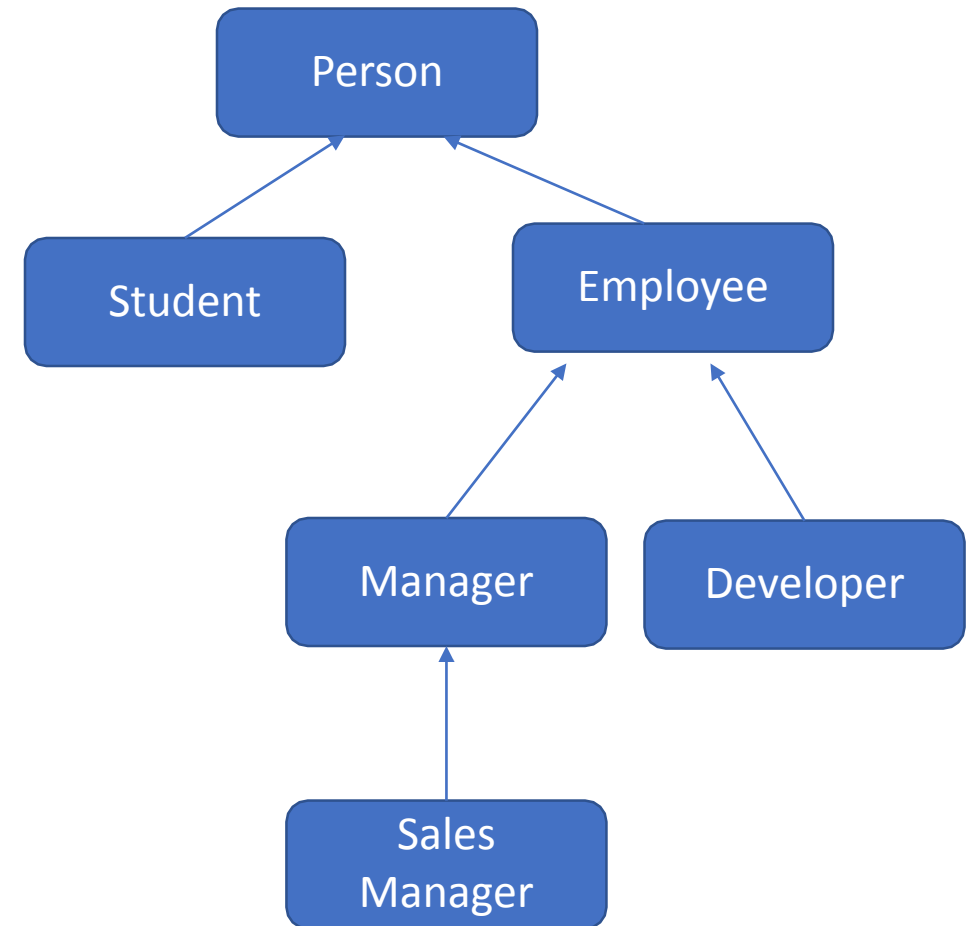
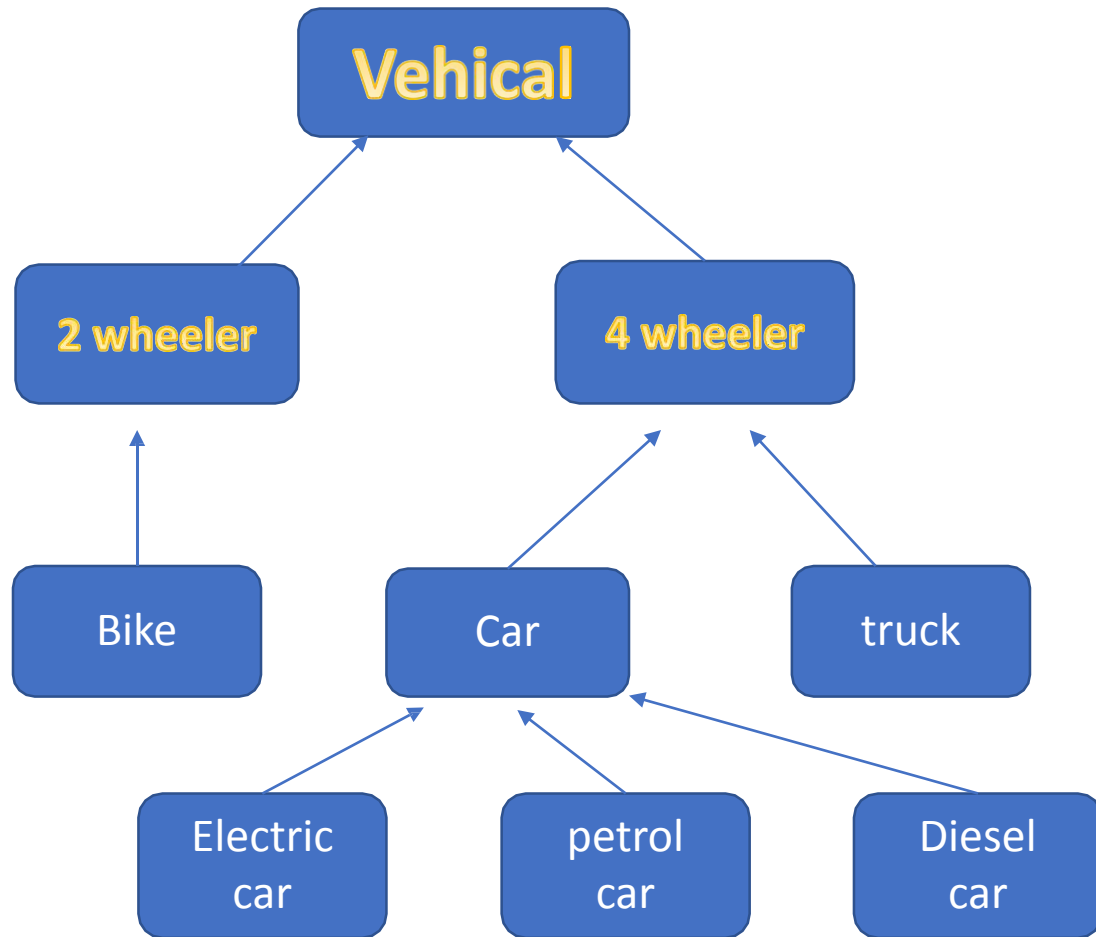
Types of Inheritance

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance

If we combine any two or more types together then it is called as hybrid inheritance.



Inheritance is also called as " journey from Generalization to Specialization".



BOOK

Inheritance

Product

Library

Association

Book

Rice

Inheritance

Food

Bowler

Inheritance

player

Mobile

Association

Charger



Mode of inheritance

- If we use private, protected and public keyword to manage visibility of the members of class then it is called as access specifier.
- But if we use these keywords to extends the class then it is called as mode of inheritance.
- C++ supports private, protected and public mode of inheritance. If we do not specify any mode, then default mode of inheritance is private.



Mode of inheritance

Mode of inheritance (read "--->" as becomes)

Base	Derived
------	---------

public mode:

Public --->	Public
protected --->	Protected
private --->	NA

protected mode:

Public --->	Protected
protected --->	Protected
private --->	NA

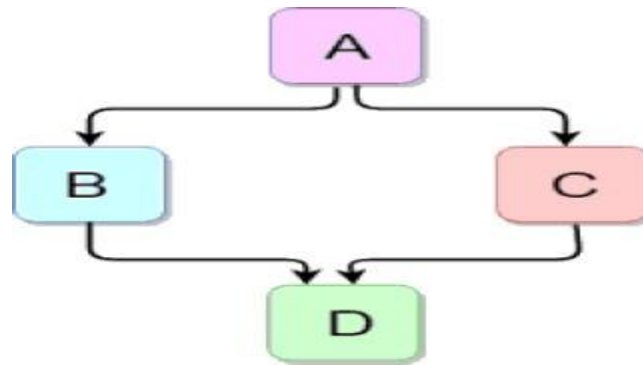
private mode:

Public --->	private
protected --->	private
private --->	NA



Diamond Problem

- As shown in diagram it is hybrid inheritance. Its shape is like diamond hence it is also called as diamond inheritance.
- Data members of indirect base class inherit into the indirect derived class multiple times.
- Member functions of indirect base class inherit into indirect derived class multiple times. If we try to call member function of indirect base class on object of indirect derived class, then compiler generates ambiguity error.
- If we create object of indirect derived class, then constructor and destructor of indirect base class gets called multiple times.
- All above problems generated by hybrid inheritance is called diamond problem.



Solution to Diamond Problem– Virtual Base Class

- If we want to overcome diamond problem, then we should declare base class virtual i.e. we should derive class B & C from class A virtually. It is called virtual inheritance. In this case, members of class A will be inherited into B & C but it will not be inherited from B & C into class D.

```
class A { };  
class B : virtual public A  
{ };  
class C : virtual public A  
{ };  
class D : public B, public C  
{ };
```



Basic Revision Of Inheritance

- If "is-a" relationship is exist between the types then we should use inheritance.
- Inheritance is also called as generalization.
- Example:
 1. Employee is a Person
 2. Manager is a Employee
 3. Book is a product
 4. Circle is a Shape
 5. SavingAccount is a account.
 6. Car is a vehicle



Inheritance

Let us consider example of employee and person:

Employee is a Person

```
class Person{    //Parent class
    //TODO
};
class Employee : public Person{ //Child class
    //TODO
};
```

- In C++, parent class is called base class and child class is called derived class.
- In above statement, "public" is called as mode of inheritance. It can be private/protected/public.
- In C++, default mode of inheritance is private.

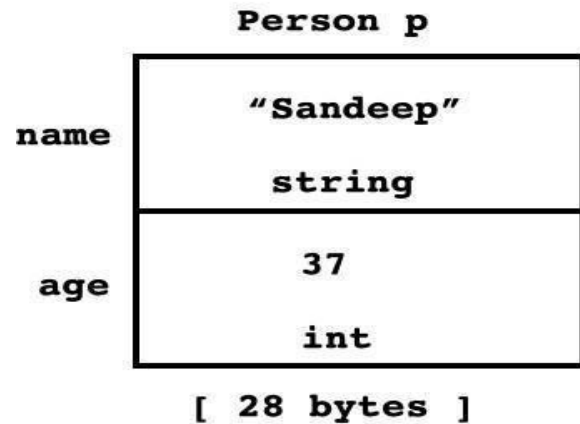


Inheritance

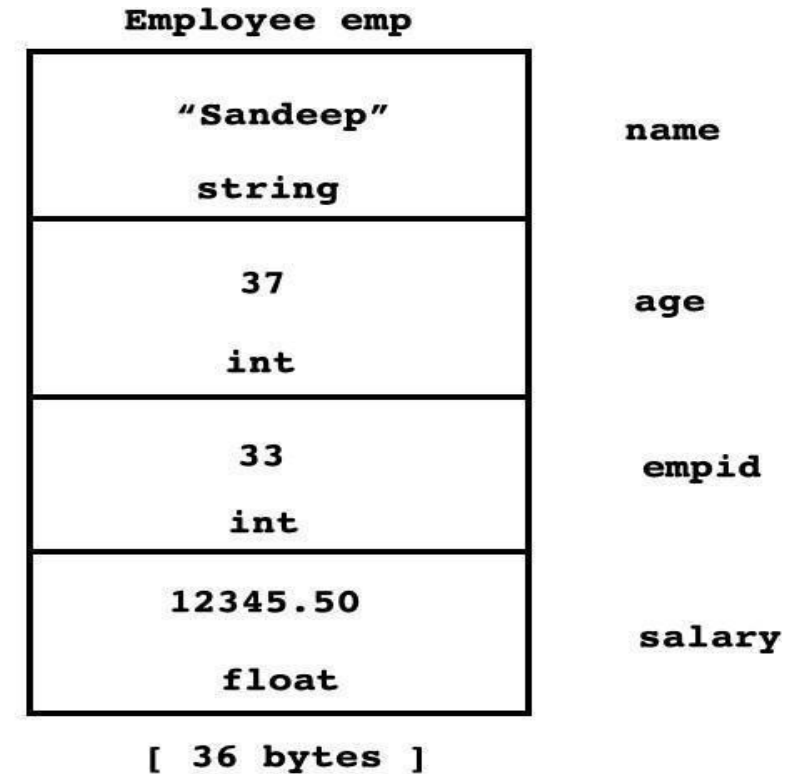
- During inheritance, members(data member / member function / nested type) of derived class, do not inherit into base class. Rather members of base class, inherit into derived class.
- All the non static data members of base class get space inside object of derived class. In other words, non static data members of base class, inherit into derived class.
- Using derived class, we can access static members of base class. In other words, static data members of base class, inherit into derived class.
- All the data members(static/non static) of base class of any access specifier(private/protected/public), inherit into derived class but only non static data member get space inside object.
- Size of object of Base class = size of of all the non static data members declared in base class.
- Size of object of Derived class = size of of all the non static data members declared in base class + size of of all the non static data members declared in derived class.



Inheritance



```
Person p("sandeep", 37);  
p.showRecord( );
```



```
Employee emp;  
emp.showRecord( );  
emp.displayRecord( );
```



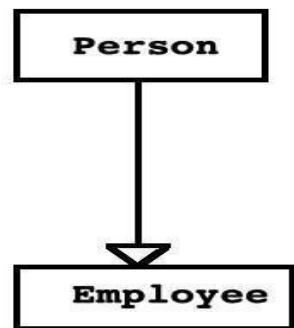
Inheritance

- We can call, non static member function of base class on object of derived class. In other words, non static member function inherit into derived class.
- We can call static member function of base class on derived class. In other words, static member function inherit into derived class.
- Following function do not inherit into derived class:
 - 1. Constructor
 - 2. Destructor
 - 3. Copy constructor
 - 4. Assignment operator function
 - 5. Friend function
- Except above 5 functions, all the member functions(static/non static) of base class inherit into derived class.

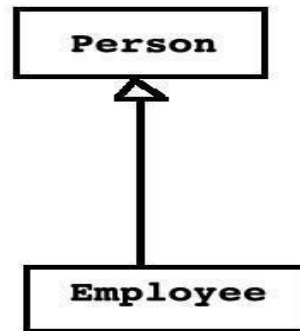


Inheritance

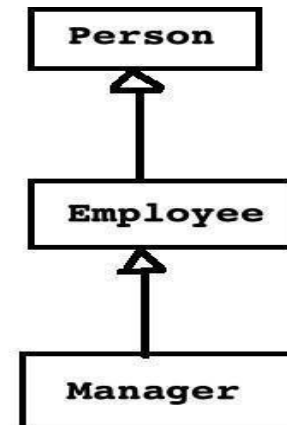
- How should we read following statement?
 - **class Employee : public Person**
 1. Class Person is inherited into class Employee.
 2. Class Employee is derived from class Person.



**Class Person is
Inherited into
Class Employee.**



**Class Employee is
Derived from
Class Person.
[Recommended]**



**Class Employee is direct Base class
And class Person is indirect Base class
Of class Manager.**

**Class Employee is direct Derived class
And class Manager is indirect derived class
Of class Person.**



Inheritance

- If we use private/protected/public keyword to control visibility of members of class then it is called access specifier.
- If we use private/protected/public keyword to extend the class / to create derived class then it is called mode of inheritance.
 1. private mode(default mode)

If has-a relationship is exist between the types then we should use either association or private mode of inheritance
 2. public mode
If is a relationship is exist between the types then we should use public mode of inheritance.
 3. protected mode



Mode of Inheritance

public mode of inheritance

Access Specifier	Same Class	Derived Class	Indirect Derived Class	Friend Function	Non Member Function
private	A	NA	NA	A	NA
protected	A	A	A	A	NA
public	A	A	A	A	A

private mode of inheritance

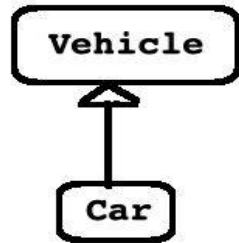
Access Specifier	Same Class	Derived Class	Indirect Derived Class	Friend Function	Non Member Function
private	A	NA	NA	A	NA
protected	A	A	NA	A	NA
public	A	A	NA	A	A Using Base class Object NA Using Derived class Object

protected mode of inheritance

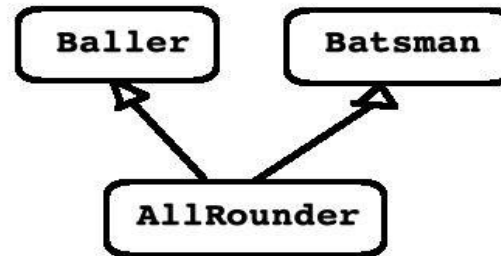
Access Specifier	Same Class	Derived Class	Indirect Derived Class	Friend Function	Non Member Function
private	A	NA	NA	A	NA
protected	A	A	A	A	NA
public	A	A	A	A	A Using Base class Object NA Using Derived class Object



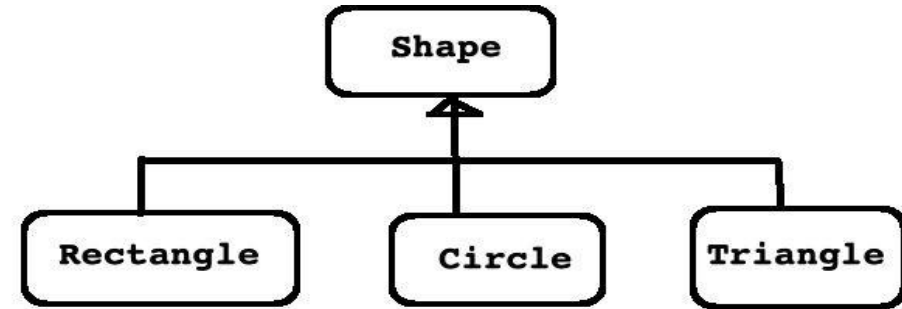
Type of Inheritance



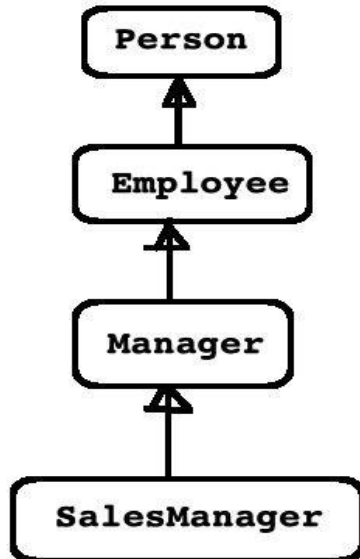
Single Inheritance



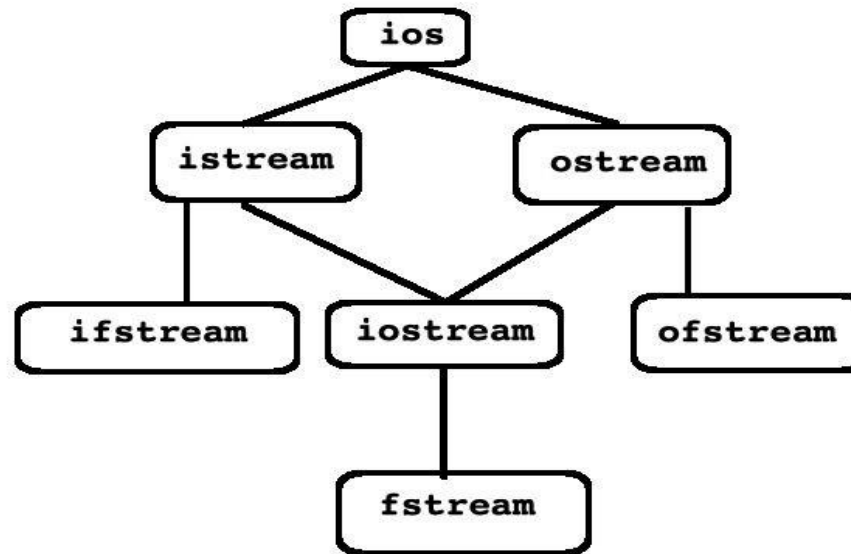
Multiple Inheritance



Hierarchical Inheritance



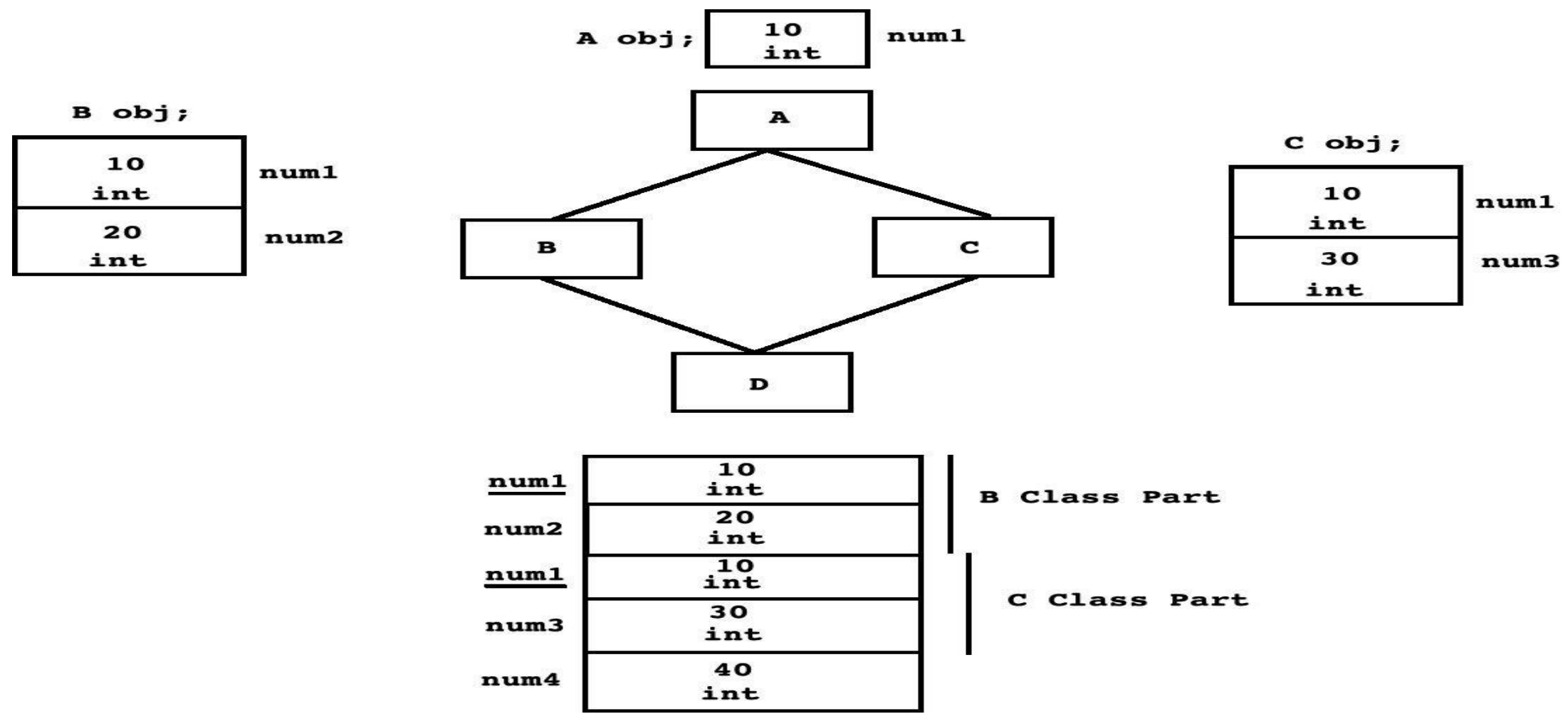
Multilevel Inheritance



Hybrid Inheritance



Diamond Problem



Diamond Problem

- Let us say Diamond Inheritance = Hierarchical Inheritance + Multiple Inheritance.
- Problems occur during diamond/hybrid inheritance:
 1. During diamond inheritance, data members of indirect base class inherit into indirect derived class multiple times. Hence it affects on size of object of indirect derived class
 2. During diamond inheritance, member function of indirect base class inherit into indirect derived class multiple times. If we try to call member function of indirect base class on object of indirect derived class, compiler generates ambiguity error.
 3. During diamond inheritance, if we create object of indirect derived class the constructor and destructor of indirect base class gets called multiple times.

• All above problems created by hybrid inheritance is called diamond problem

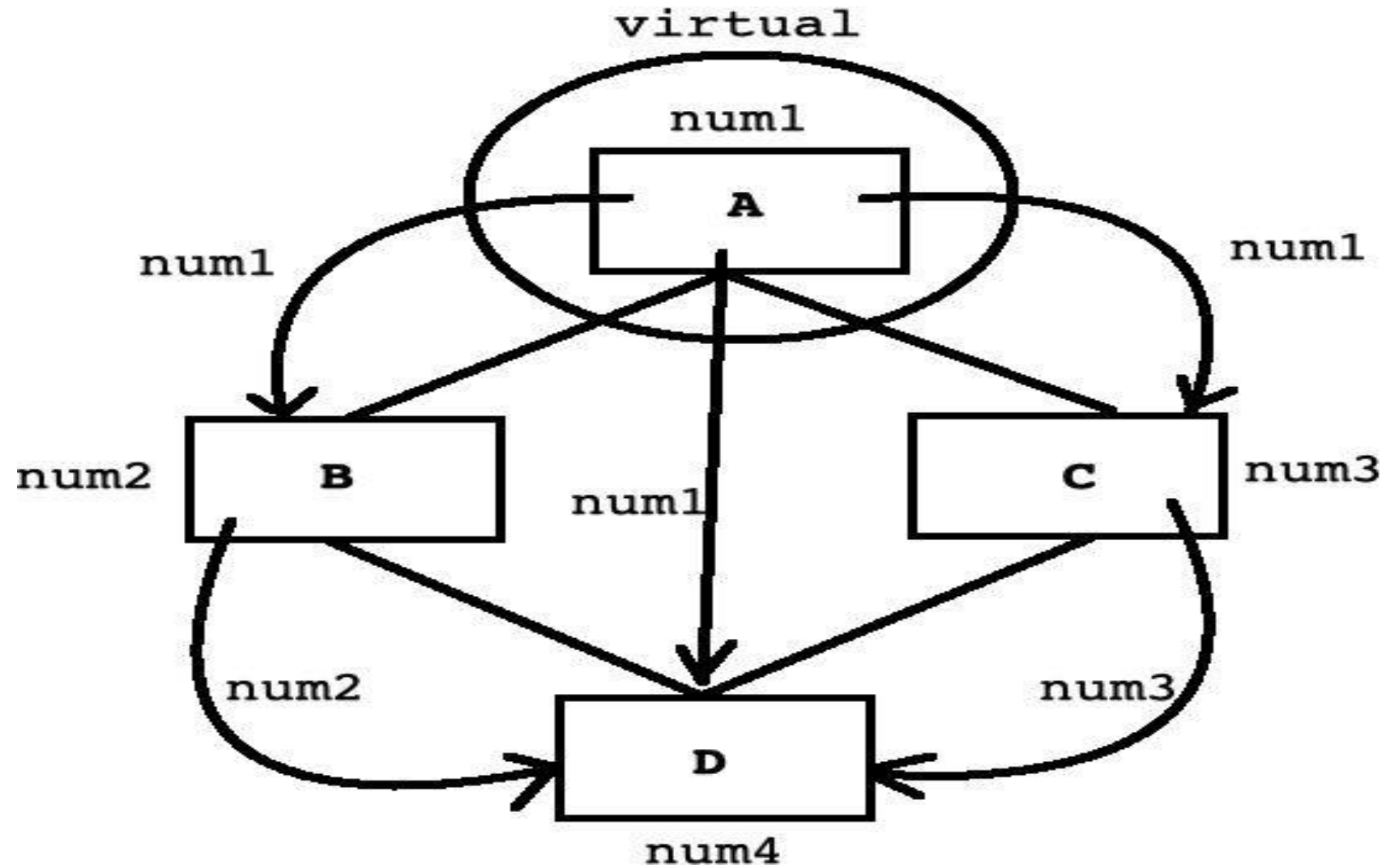
Virtual Inheritance

- virtual is keyword in C++.
- **If we want to avoid diamond problem then we should declare base class virtual.**

```
class A{    };  
  
class B : virtual public A{ };    //Virtual Inheritance  
  
class C : virtual public A{ };    //Virtual Inheritance  
  
class D : public B, public C{ };
```



virtual Inheritance



Inheritance

- If we create object of base class then only base class constructor and destructor gets called.
- If we create object of derived class then first base class constructor gets called and then derived class constructor gets called. Destructor calling sequence is exactly opposite.
- From any constructor of derived class, by default, base class's parameterless constructor gets called.
- If we want to call, any constructor of base class from constructor of derived class then we should use constructor's base initializer list.



Inheritance

```
class Base{
    int num1;
public:
    Base( int num1 ){
        this->num1 = num1;
    }
};

class Derived : public Base {
    int num2;
public:
    Derived( int num1, int num2 ) : Base( num1 ) //constructor's base initializer list {
        this->num2 = num2;
    }
};

Derived d( 100, 200 );
```



Shadowing

If implementation of base class member function is logically incomplete then we should redefine member function inside derived class. In other words, we should give same name to the member function in derived class

- If name of base class and derived class member function is same and if we try to call such member function on object of derived class then preference is given to the derived class member function. Here derived class member function hides implementation of inherited function. This process is called **shadowing**
- Without changing implementation of existing class, if we want to extend meaning of that class then we should use inheritance.



Shadowing

```
class Person{
public:
    void print( void ){
        cout<<"Person::print"<<endl;
    }
};

class Employee : public Person{
public:
    void print( void ){
        cout<<"Employee::print"<<endl;
    }
};

int main( void ){
    Employee emp;
    emp.Person::print( );           //Person::print
    emp.Employee::print( );         //Employee::print
    emp.print( );                   //Employee::print
    return 0;
}
```



Inheritance

During inheritance, members(data member, member function, nested types) of derived class do not inherit into base class. Hence using base class object, we can access members of base class only.

- During inheritance, members(data member, member function, nested types) of class object, we Base class inherit into derived class. Hence using derived can access members of base class as well as derived class.
- Members of base class inherit into derived class. Hence we can consider object of derived class as a object of base class.
- Since derived class object can be considered as base class object, we can use it in place of base class object.



Upcasting and downcasting

- Upcasting and downcasting :-

Upcasting - process of converting derived class pointer into base class pointer
or Storing address of derived class object into base class pointer.

eg : `Person *p=new student();`

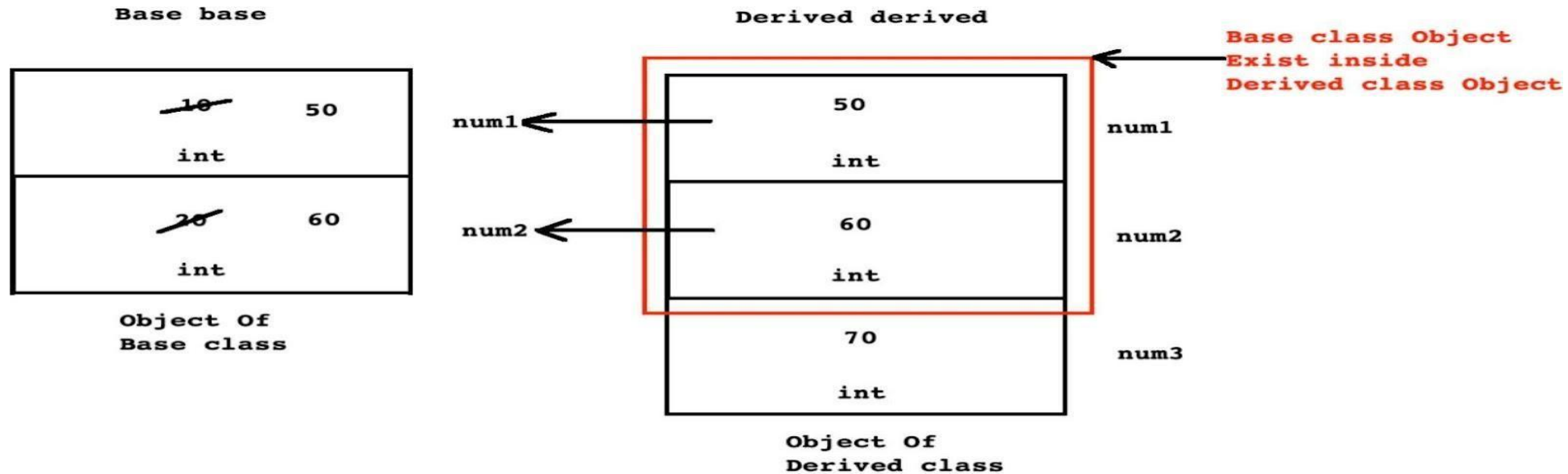
Downcasting - process of converting base class pointer into derived class pointer
or storing address of base class object into derived class pointer

eg : `Student *s=new person();`



Object Slicing

- If we assign, derived class object to the base class object then compiler consider only base class portion from derived class object and copy it into base class object. This process is called object slicing
- During object slicing, node of inheritance must be public.



[Object Slicing]



Virtual Keyword

- **Virtual function** = It is the function which is called depending on type of object rather than type of pointer
- If class contains at least one virtual function then such class is called **polymorphic class**.
- If signature of base class and derived class member function is same and if function in base class is virtual then derived class member function is by default considered as virtual.



Virtual Function

- In case of upcasting, If we want to call function depending on type of pointer then we should declare function in base object rather than type of class virtual.
- If class contains at least one virtual function then such class is called polymorphic class
- If signature of base class and derived class member function is same and if derived class member function is by function in base class is virtual then default considered as virtual.
- Process of redefining, virtual function of base class, inside derived class with same signature is called function overriding.
- Virtual function redefined function.
- For function overriding:
 1. Functions
 2. Signature
 3. At least, inside derived class is called overridden must be exist inside base class and derived class. of base class and derived function must be same. function in base class must be virtual



Function overriding.

Process of redefining, virtual function of base class, inside derived class with same signature is called function overriding.

- Virtual function redefined inside derived class is called overridden function.
- For function overriding:
 1. Functions must exist inside base class and derived class.
 2. Signature of base class and derived function must be same.
 3. Function in base class must be virtual



Program Demo

Early Binding

create a class Base and Derived (void show() in both classes)

```
create base *bptr;
```

```
bptr=&d;
```

```
bptr->show(  
)
```

Late Binding

create a class Base and Derived (void show() in both classes one as virtual in base class)

```
create base *bptr;
```

```
bptr=&d;
```

```
bptr->show(  
)
```



Pure virtual function and Abstract class

- ❑ Virtual fun which is equated to zero such function is called as Pure virtual function
- ❑ Pure virtual function does not have body.
- ❑ A class which contains at least one Pure virtual function such class is called as "Abstract class".
- ❑ If class is Abstract we can not create object of that class but we can create pointer or reference of that class .
- ❑ It is not compulsory to override virtual function but It is compulsory to override Pure virtual function
- ❑ If we not override pure virtual function in derived class at that time derived class can be treated as abstract class.



```
namespace std {  
    class type_info  
    {  
    private:  
        type_info(const type_info& other);  
        type_info& operator=(const type_info& other);  
    public:  
        const char* name() const throw( );  
        bool operator==(const type_info& rhs) const throw();  
        bool operator!=(const type_info& rhs) const throw();  
        virtual ~type_info();  
    };  
}
```



Inheritance

type_info is a class declared in std namespace and std namespace is declared in header file.

- RTTI is a process of getting information(data type name) of any object at runtime.
- If we want to use RTTI in a program then we should use typeid operator.
- typeid operator returns reference of constant object of type_info class which contains type information.

```
#include<iostream>
#include<typeinfo>
using namespace std;
int main( void ){
    int number = 0;
    const type_info &type = typeid( number );
    const char* typeName = type.name( );
    cout<<"Type Name      :      "<<typeName<<endl;
    return 0;
}
```



Steps to use RTTI:

1. Include header file(`# include`)
 2. Use `typeid()` operator
 3. Use `name()` function.
- In case of upcasting if we want to find out type of object instead of type of pointer then we should use RTTI.
 - In case of upcasting, using RTTI, if we want to find out true type of object then base class must be polymorphic.
 - Using NULL pointer, if we try to find out type of object then `typeid()` operator throws `bad_typeid` exception.



1. static_cast

In case of non-polymorphic type, to do down casting, use static_cast

2. dynamic_cast

In case of polymorphic type, to do down casting, use dynamic_cast

3. const_cast

To convert pointer to constant object into pointer to non constant object / reference to constant object into reference to non constant object, use const_cast operator.

4. reinterpret_cast

To convert pointer of any type into pointer of any other type use reinterpret_cast operator.



Difference between Procedure Oriented and Object Oriented

Procedure Oriented

- Emphasis on steps or algo
- Programs are divided into small code units called Functions.
- Most function shares global data and can modify it
- Data moves from function to function
- Follows Top-down approach
- Example= C

Object Oriented

- Emphasis on data of program
- Programs are divided into small data units called classes
- Data is hidden and not accessible outside the class
- Objects are communicating with each other
- Follows Bottom-up approach
- Example =C++,JAVA,C#.NET, python

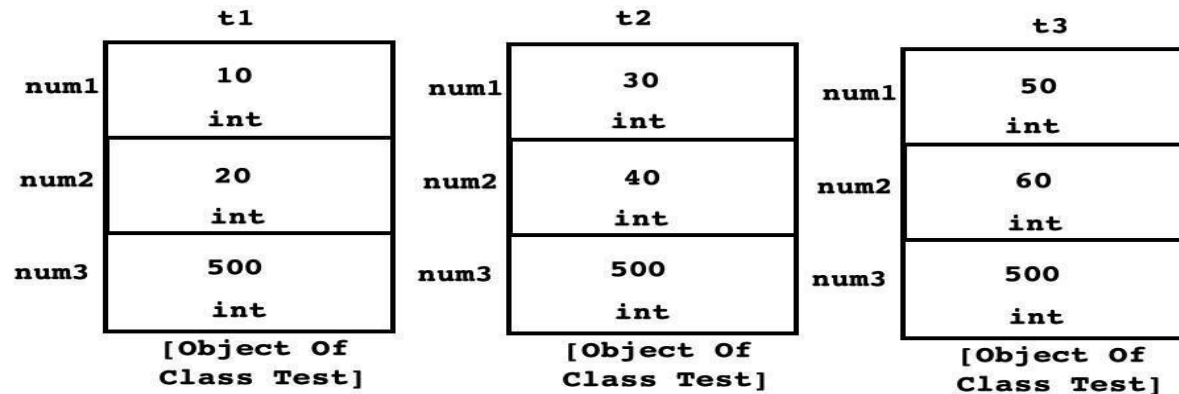


Static Data member

- Member function(static/non static) do not get space inside object. Hence size of object depends on size of all the data members declared inside class.

```
class Test{
private:
    int num1;
    int num2;
    int num3;
public:
    Test( void ){
        this->num1 = 0;
        this->num2 = 0;
        num3 = 500;
    }
    Test( int num1, int num2 ){
        this->num1 = num1;
        this->num2 = num2;
        num3 = 500;
    }
};

int main( void ){
    Test t1( 10, 20 );
    Test t2( 30, 40 );
    Test t3( 50, 60 );
    return 0;
}
```



Static Data member

- **Instance Variable** : A data member of a class which get space inside object is called instance variable. In short, non static data member declared inside class is called instance variable.
- **Instance Method** : A member function of a class, which is designed to call on object is called instance method. In short, non static member function defined inside class is called instance method.
- **Concrete Method** : A member function of a class, which is having body is called concrete method.
- **Concrete Class** : A class from which we can create object is called concrete class. In other words, we can instantiate concrete class.
- **Revision of some oops concepts:**
 - Data member declared inside class get space once per object according to their declaration inside class.
 - If we call non static member function on object then non static member function get this pointer.
 - With the help of this pointer, all the objects of same class share single copy of member function.

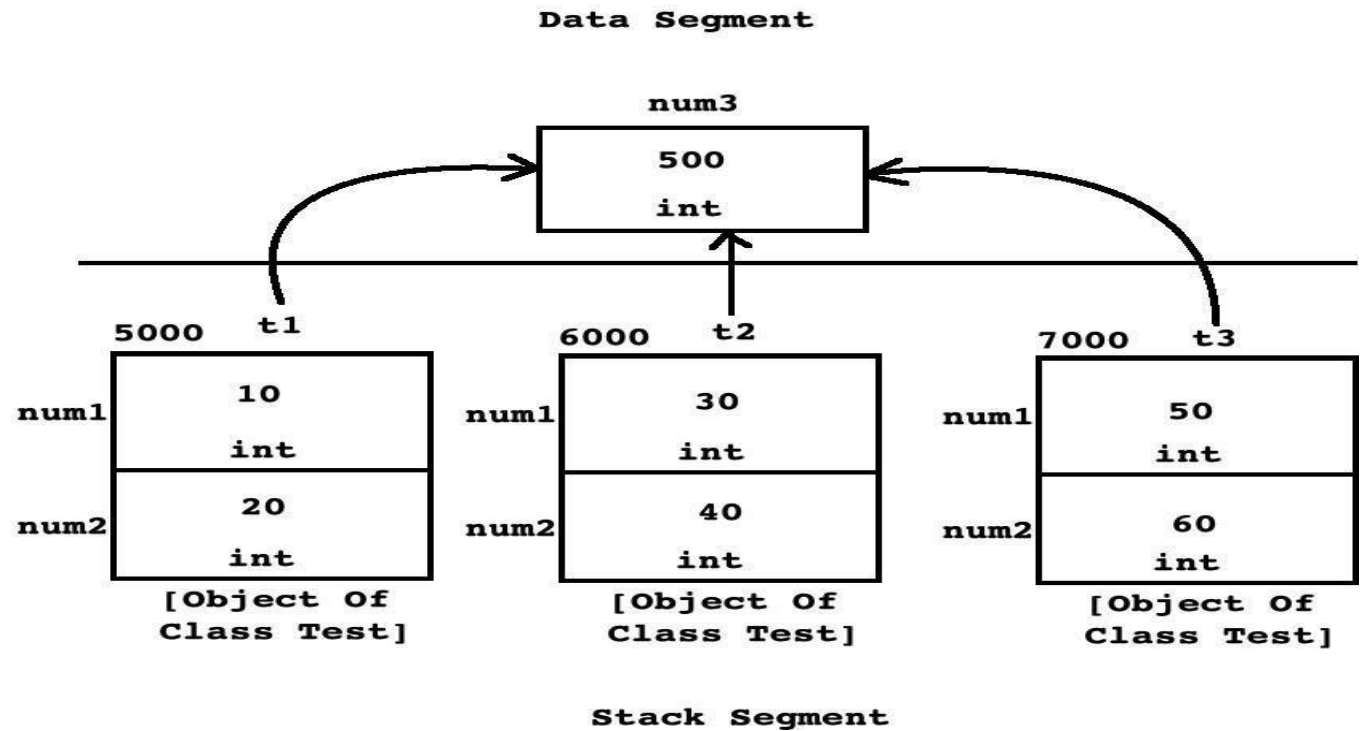
•

Static Data member

- If we want to share value of any data member inside all the objects of same class then we should declare data member static.
-

```
class Test{
private:
    int num1; //Instance Variable
    int num2; //Instance Variable
    static int num3; //Class level Variable
public:
    Test( void ){
        this->num1 = 0;
        this->num2 = 0;
    }
    Test( int num1, int num2 ){
        this->num1 = num1;
        this->num2 = num2;
    }
    //Test *const this = &t1
    void printRecord( void ){
        cout<<"Num1 : "<<this->num1<<endl;
        cout<<"Num2 : "<<this->num2<<endl;
        cout<<"Num3 : "<<Test::num3<<endl;
    }
};

int Test::num3 = 500; //Global Definition
int main( void ){
    Test t1( 10, 20 );
    t1.printRecord( );
    return 0;
}
```



Static Data member

- Static data member do not get space inside object. Rather all the objects of same class share single copy of it. Hence size of object is depends on size of all the non static data members declared inside class.
- A data member of a class which get space inside object is called instance variable.
- Instance variable is designed to access using object or pointer/reference of object.
- A data member of a class which do not get space inside object is called class level variable.
- Class level variable is designed to access using class name and scope resolution operator.
- If we declare data member static then we must provide global definition for it. Otherwise linker will generate error.
- Non static data member get space once per object. But static data member get space once per class.

Static Data member

- Only non static data member get space inside object. To initialize object we define constructor inside class. In other words, we should use constructor to initialize non static data member which get space inside object.
- Since static data member do not get space inside object, we should not initialize static data member inside constructor.

Static member function

- In C++, we can declare global function as well as member function static.
- If we want to access non static members of the class then member function should be non static and if we want to access static members of the class then member function should be static.
- Non static member function get this pointer. Hence inside non static member function, we can access static as well as non static members.
- Why static member function do not get this pointer?
 - If we call non static member function on object then non static member function get this pointer. In other words, non static member functions are designed to call on object.
 - Static member function is designed to call on class name.
 - **Since static member function is not designed to call on object, it doesn't get this pointer.**

Static member function

- Since static member function do not get this pointer, we can not access non static members inside static member function. In other words, static member function, can access only static members of the class directly.
- Using object, we can use non static members inside static member function.
- **Inside member function, if we are going to use this pointer then member function should be non static otherwise it should be static.**
- **We can not declare static member function:**
 1. **constant**
 2. **volatile**
 3. **Virtual**

Static member function

- Since static member function do not get this pointer, we can not access non static members inside static member function. In other words, static member function, can access only static members of the class directly.
- Using object, we can use non static members inside static member function.
- **Inside member function, if we are going to use this pointer then member function should be non static otherwise it should be static.**
- **We can not declare static member function:**
 1. **constant**
 2. **volatile**
 3. **Virtual**

Template

- If we want to write generic program in C++, then we should use template.
- This feature is mainly designed for implementing generic data structure and algorithm.
- If we want to write generic program, then we should pass data type as a argument. And to catch that type we should define template.
- Using template we can not reduce code size or execution time but we can reduce developers effort.



Template

```
int num1 = 10, num2 = 20;
swap_object<int>( num1, num2 );
string str1="Pune", str2="Karad";
swap_object<string>( str1, str2 );
```

In this code, <int> and <string> is considered as type argument.

```
template<typename T> //or
template<class T> //T : Type
Parameter void swap( b obj1, T obj2 )
{
    T temp = obj1;
    obj1 = obj2;
    obj2 = temp;
}
```

template and typename is keyword in C++. By passing datatype as argument we can write generic code hence parameterized type is called template

• Types of Template

- Function Template
- Class Template



Example of Function Template

```
//template<typename T> //T : Type Parameter
template<class T> //T : Type Parameter

void swap_number( T &o1, T &o2 )

{   T temp = o1;

    o1 = o2;

    o2 = temp;
}

int main( void )
{
    int num1 = 10;

    int num2 = 20;

    swap_number<int>( num1, num2 ); //Here int is type argument
    cout<<"Num1 : "<<num1<<endl;

    cout<<"Num2 : "<<num2<<endl;

    return 0;
}
```



Example of Class Template

```
template<class T>
class Array // Parameterized type
{
private:
    int size;
    T *arr;
public:
    Array( void ) : size( 0 ), arr( NULL )
    {
    }
    Array( int size )
    {
        this->size = size;
        this->arr = new T[ this->size ];
    }
    void acceptRecord( void ){}
    void printRecord( void ){ }
    ~Array( void ){ }
};
```

```
int main(void)
{
    Array<char> a1( 3 );
    a1.acceptRecord();
    a1.printRecord();
    return 0;
}
```



Thank You

